

TECHNICAL UNIVERSITY OF MOLDOVA

Faculty of Computers, Informatics and Microelectronics

Software Engineering

OPERATING SYSTEMS

Laboratory Work No. 9

Watching System Calls with strace

Performed by:
Bianca Tataroi
Group FAF-242

Checked by:
Patricia Reițman

Chisinau, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Part 1 - Observing System Calls **2**
3.1 Execution 2
3.2 Results 3
3.3 Observations 4

4 Part 2 - Direct System Calls in C **4**
4.1 Implementation 4
4.2 Compilation and Execution 4
4.3 Monitoring with strace 5
4.4 File Descriptors 6

5 Part 3 - Counting System Calls **6**
5.1 Execution 6
5.2 Results 6
5.3 Observations 7

6 Conclusion **7**

7 GitHub Repository **7**

1 Introduction

System calls provide an interface between user-space programs and the operating system kernel.

A program uses system calls to request operating system services, such as opening files, reading data, writing information, and closing files.

In this laboratory work, the Linux tool `strace` is used to monitor system calls made by programs.

The laboratory also demonstrates how a C program interacts directly with the Linux kernel through file-related system calls.

2 Objectives

The main objectives of this laboratory work are:

- Understand the role of system calls in Linux.
- Use `strace` to observe the system calls of a Linux command.
- Implement a C program using `open()`, `write()`, `read()`, and `close()`.
- Identify the purpose of file descriptors.
- Count system calls using `strace -c`.

3 Part 1 - Observing System Calls

3.1 Execution

The first task was to observe the system calls performed by the Linux `ls -l` command.

The following command was executed:

```
strace -f -e trace=openat,read,write,close ls -l
```

The `strace` tool displays the system calls executed by a program.

The `-f` option traces child processes, while the `-e trace` option limits the displayed calls to the selected types.

3.2 Results

```
bianca@ubuntu-lab:~/os-lab9$ strace -f -e trace=openat,read,write,close
ls -l
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libselinux.so.1", O_RDONLY|
O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\0\0\0\0\0\0
\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_
CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\0\0\0\0\0\0
\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libm.so.6", O_RDONLY|O_CLOE
XEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\0\0\0\0\0\0
\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOE
XEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\3101\2\0\0\0\
0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libpcre2-8.so.0", O_RDONLY|
O_CLOEXEC) = 3
```

Figure 1: Initial system calls executed by ls -l

The output shows multiple `openat()`, `read()`, and `close()` calls.

These calls are related to accessing files and loading libraries required for execution.

```
openat(AT_FDCWD, "/usr/share/coreutils/locales/uucore/en-US.ftl", O_RDON
LY|O_CLOEXEC) = 3
read(3, "# Common strings shared across a"..., 4080) = 4080
read(3, "", 32)                         = 0
close(3)                                = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/ls/en-US.ftl", O_RDONLY|O
_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
close(3)                                = 0
write(1, "total 0\n", 8total 0
)                 = 8
+++ exited with 0 +++
```

Figure 2: Additional system calls and program termination

The second screenshot shows a call to `write()` and the successful termination of the command.

The message `exited with 0` indicates that the program finished successfully.

3.3 Observations

Question: Which system call does `ls` use to print information on the screen?

Answer:

The `ls` command uses the `write()` system call.

It normally writes to file descriptor 1, which represents standard output (`stdout`).

The trace also shows that programs may access several system libraries before performing their main operations.

4 Part 2 - Direct System Calls in C

4.1 Implementation

A C program named `syscalls.c` was created to demonstrate direct file-related system calls.

The program performs the following operations:

1. Opens a file named `note.txt`.
2. Writes the message `hello, kernel`.
3. Closes the file.
4. Opens the file again for reading.
5. Reads the stored message.
6. Displays the message and closes the file.

The program uses the system call interfaces `open()`, `write()`, `read()`, and `close()`.

4.2 Compilation and Execution

The program was compiled using GCC:

```
gcc -Wall -Wextra -o syscalls syscalls.c
```

It was then executed using:

```
./syscalls
```

The generated file was verified with:

```
cat note.txt
```

```
bianca@ubuntu-lab:~/os-lab9$ cat note.txt
hello, kernel
```

Figure 3: Verification of the content of note.txt

The file contains the expected message:

```
hello, kernel
```

The complete program output was:

```
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
```

This confirms that the program successfully wrote and read 14 bytes.

4.3 Monitoring with strace

The following command was executed:

```
strace -e trace=openat,write,read,close ./syscalls
```

```
bianca@ubuntu-lab:~/os-lab9$ strace -e trace=openat,write,read,close ./s
yscalls
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOE
XEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\3101\2\0\0\0\
0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(1, "open returned descriptor 3\n", 27open returned descriptor 3
) = 27
write(3, "hello, kernel\n", 14)         = 14
write(1, "write sent 14 bytes\n", 20write sent 14 bytes
)   = 20
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
write(1, "read got 14 bytes: hello, kernel"..., 33read got 14 bytes: hel
lo, kernel
) = 33
close(3)                                = 0
+++ exited with 0 +++
```

Figure 4: System calls generated by syscalls.c

The trace shows that the C function `open()` is implemented through the Linux `openat()` system call.

The program creates `note.txt`, writes 14 bytes, closes the descriptor, and opens the file again to read the content.

The `read()` call returns 14, which represents the number of bytes read.

4.4 File Descriptors

The first file descriptor returned by `open()` is 3.

This happens because the first three descriptors are normally reserved for standard input, output, and error.

Descriptor	Purpose
0	Standard input (stdin)
1	Standard output (stdout)
2	Standard error (stderr)
3	First normally available file descriptor

Table 1: Standard Linux file descriptors

5 Part 3 - Counting System Calls

5.1 Execution

The final part investigated the number of system calls made by the C program.

The following command was executed:

```
strace -c -e trace=openat,read,write,close ./syscalls
```

The `-c` option displays a summary containing the number of calls, execution time, and errors for each system call.

5.2 Results

```
bianca@ubuntu-lab:~/os-lab9$ strace -c -e trace=openat,read,write,close
./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 92.42    0.000061          30         2           read
  4.55    0.000003           0         4           write
  3.03    0.000002           0         4           close
  0.00    0.000000           0         4           openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.000066           4        14           total
```

Figure 5: Summary of system calls using strace -c

The results obtained during execution are presented in Table 2.

System Call	Number of Calls
openat	4
read	2
write	4
close	4
Total	**14**

Table 2: System call counts obtained using strace

5.3 Observations

The program executed 14 system calls of the four selected types.

The `openat()` calls were used to open files, including the required system library and `note.txt`.

The `read()` calls retrieved data from files.

The `write()` calls included writing the message into the file and printing results to standard output.

The `close()` calls released file descriptors.

These results demonstrate that even a small C program interacts with the operating system multiple times.

6 Conclusion

During this laboratory work, the `strace` utility was used to observe and analyze Linux system calls.

In Part 1, the system calls executed by the `ls -l` command were examined.

In Part 2, a C program was implemented to open, write, read, and close a file.

The program successfully wrote and read the message `hello, kernel`, containing 14 bytes.

In Part 3, the `strace -c` option was used to count system calls.

The results showed a total of 14 selected system calls during the execution of the C program.

This laboratory helped demonstrate how user-space programs communicate with the Linux kernel through system calls.

7 GitHub Repository

The laboratory source code and supporting files are available in the following repository:

https://github.com/Biancatataroi/Operating-Systems-Lab9